

2 Calculations Homework

Sai Ram Vodnala

06/06/2024

General instructions.

There are four exercises below. Three exercises are worth 10 points each, and are labelled (Either R, Python or Julia). For these exercises, you can implement a solution in the language of your choice. I've included code chunk templates for all three languages, but you only need to fill in the chunks for one language.

One exercise is labelled (R, Python and Julia). This exercise is worth 20 points, and you must provide a solution using all three languages. This exercise also includes code chunk templates for all three languages.

You should not write functions for these exercises, we will be working with functions in the next set of exercises. These exercises are all about coding correct arithmetic.

Some of the exercises have questions relating to the results. Write your answers in the narrative part of the text, formatted as *italicized* text, **bold** text or

quoted text.

Change the name of this file to match your user name on D2L, keeping the 'Rmd' extension, and include week number in the title (for example, **Peter.Claussen.2.Rmd**). Upload this file to D2L. Typeset this file to Word or PDF and upload the result to D2L as well.

Exercise 1 (Either R, Python or Julia)

Using values from Table, show how Wansink and Payne calculate percent increase in calories per recipe (43.7%), calories per serving (63.0%) and servings per recipe (-2.0). That is, confirm the values in the final column of Table 1 (Change from 1936 to 2006, %, p. 292). Can you duplicate their results?

You do not need to define variables for this exercise, simply enter the calculations in R, Python or Julia syntax, and show the results. You are free, however, to define variables.

Answer

Calories per Recipe

```
calories_per_recipe_1936 <- 2123.8
calories_per_recipe_2006 <- 3051.9
percent_increase_calories_recipe <- ((calories_per_recipe_2006 - calories_per_recipe_1936) / calories_p
print(percent_increase_calories_recipe)
```

R

```
## [1] 43.69997
```

Python

Julia

Calories per Serving

```
calories_per_serving_1936 <- 268.1
calories_per_serving_2006 <- 384.4
percent_increase_calories_serving <- ((calories_per_serving_2006 - calories_per_serving_1936) / calories_per_serving_1936)
print(percent_increase_calories_serving)
```

R

```
## [1] 43.37934
```

Python

Julia

Servings per Recipe

```
serving_per_recipe_1936 <- 12.9
serving_per_recipe_2006 <- 12.7
percent_increase_serving_recipe <- ((serving_per_recipe_2006 - serving_per_recipe_1936) / serving_per_recipe_1936)
print(percent_increase_serving_recipe)
```

R

```
## [1] -1.550388
```

Python

Julia

Exercise 2 (Either R, Python or Julia)

Using the values from Wansink, Table 1 as given in Homework 1 or in the course outline, calculate p -values for the difference between calories per serving for each of the pairs of means listed below:

- 1936 versus 2006
- 1936 versus 1997
- 1997 versus 2006

You may use either Wald's test or Student's t .

How do your calculated p -values compare to the F test p -value reported in Table 1. Does this test provide stronger or weaker evidence for the claim that calories per serving has changed over time?

Answer

The F-test p-value reported in Table 1 is 0.046, which is larger than the p-values calculated using the t-test. So, This shows that the t-test provides stronger evidence for the claim that calories per serving has changed over time compared to the F-test. The smaller the p-value, the stronger the evidence against the null hypothesis. Therefore, the t-test suggests stronger evidence for a difference in calories per serving over time.

Enter the R, Python or Julia code in the chunks below.

1936 versus 2006

```
calories_per_serving_SD_1936 <- 124.8; calories_per_serving_SD_2006 <-168.3;

t <-(calories_per_serving_2006 - calories_per_serving_1936)/sqrt((calories_per_serving_SD_1936^2/18)+(c
print(t)
```

R

```
## [1] 2.354963
```

```
2*(1-pt(abs(t),2*(18-1)))
```

```
## [1] 0.02443665
```

Python

Julia

1936 versus 1997

```
calories_per_serving_1997 <- 288.6; calories_per_serving_SD_1997 <- 122.0;
t1 <-(calories_per_serving_1997 - calories_per_serving_1936)/sqrt((calories_per_serving_SD_1936^2/18)+(
print(t1)
```

R

```
## [1] 0.4983472
```

```
2*(1-pt(abs(t1),2*(18-1)))
```

```
## [1] 0.621448
```

Python

Julia

1997 versus 2006

```
t2 <-(calories_per_serving_2006 - calories_per_serving_1997)/sqrt((calories_per_serving_SD_2006^2/18)+(
print(t2)
```

R

```
## [1] 1.955308
```

```
2*(1-pt(abs(t2),2*(18-1)))
```

```
## [1] 0.0588065
```

Python

Julia

Exercise 3 (Either R, Python or Julia)

Part a

Write code to compute

$$7 - 1 \times 0 + 3 \div 3$$

Type this in verbatim, using only numbers, -,*, and /, with no parenthesis. Do you agree with the result?

Answer

I agree with the result because of Using the order of (PEMDAS/BODMAS) operations, Generally we perform the division and multiplication first. Next we perform the addition and subtraction. So the answer is 8 but when we use parenthesis we will get different answers because of the position of parenthesis.

```
result <- 7 - 1 * 0 + 3 / 3
result
```

R

```
## [1] 8
```

Python

Julia

Part b

According to “Why Did 74% of Facebook Users Get This Wrong?” (<https://profpete.com/blog/2012/11/04/why-did-74-of-facebook-users-get-this-wrong/>), most people would compute the result as 1. Use parenthesis () to produce this result.

Answer

I think it is because of parenthesis. we will get different answers due to parenthesis position. The below chunk where I have added parenthesis and the result of equation has changed to 1.

```
result <- ((7 - 1) * 0) + (3 / 3)
result
```

R

```
## [1] 1
```

Python

Julia

Part c

Several respondents to the survey cited in Part 2 gave the answer 6. Add parenthesis to produce this result.

Answer

This is the another response if we change the position of the parenthesis.

```
result <- 7 - ((1 * 0) + (3 / 3))
result
```

R

```
## [1] 6
```

Python

Julia

Exercise 4 (R, Python and Julia)

The probability density function for the distribution of a random variable denoted x , when taken from a normal population with mean μ and variance σ^2 is calculated by

$$f(x, \mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

For values of $x = \{-1.0, 0.0, 1.0\}$, write code to implement the formula above, for $\mu = 1$ and $\sigma = 2$.

Answer

Enter the R code in the chunks below.

$x = -1$

```
x <- -1.0;
mu <- 1 ;sigma <- 2;
f <- (1/(sigma*sqrt(2*pi)))*exp(-((x-mu)^2)/(2*sigma^2))
print(f)
```

R

```
## [1] 0.1209854
```

```
import math
import scipy.stats as stats
x = -1.0
mu = 1
sigma = 2
f = (1 / (sigma * math.sqrt(2 * math.pi))) * math.exp(-((x - mu) ** 2) / (2 * sigma ** 2))
print(f)
```

Python

```
## 0.12098536225957168
```

```
x = -1.0
```

Julia

```
## -1.0
```

```
mu = 1
```

```
## 1
```

```
sigma = 2
```

```
## 2
```

```
p_value = 1 / (sigma * sqrt(2*pi)) * exp(-((x - mu)^2) / (2 * sigma^2))
```

```
## 0.12098536225957168
```

```
x = 0.0
```

```
x <- 0.0;
mu <- 1 ;sigma <- 2;
f <- (1/(sigma*sqrt(2*pi)))*exp(-((x-mu)^2)/(2*sigma^2))
print(f)
```

R

```
## [1] 0.1760327
```

```
import math
import scipy.stats as stats
x = 0.0
mu = 1
sigma = 2

f = (1 / (sigma * math.sqrt(2 * math.pi))) * math.exp(-((x - mu) ** 2) / (2 * sigma ** 2))
print(f)
```

Python

```
## 0.17603266338214976
```

```
x = 0.0
```

Julia

```
## 0.0
```

```
mu = 1
```

```
## 1
```

```
sigma = 2
```

```
## 2
```

```
p_value = 1 / (sigma * sqrt(2pi)) * exp(-((x - mu)^2) / (2 * sigma^2))
```

```
## 0.17603266338214976
```

```
x = 1
```

```
x <- 1.0;
mu <- 1 ;sigma <- 2;
f <- (1/(sigma*sqrt(2*pi)))*exp(-((x-mu)^2)/(2*sigma^2))
print(f)
```

R

```
## [1] 0.1994711
```

```
import math
import scipy.stats as stats
x = 1.0
mu = 1
sigma = 2

f = (1 / (sigma * math.sqrt(2 * math.pi))) * math.exp(-((x - mu) ** 2) / (2 * sigma ** 2))
print(f)
```

Python

```
## 0.19947114020071635
```

```
x = 1.0
```

Julia

```
## 1.0
```

```
mu = 1
```

```
## 1
```

```
sigma = 2
```

```
## 2
```

```
p_value = 1 / (sigma * sqrt(2pi)) * exp(-((x - mu)^2) / (2 * sigma^2))
```

```
## 0.19947114020071635
```

You can confirm your results using the built in normal distribution functions for each language, as described in lecture.

```
dnorm(-1.0,1,2)
dnorm(0,1,2)
dnorm(1,1,2)
```

R

```
import scipy

scipy.stats.norm.pdf(-1.0,1,2)
scipy.stats.norm.pdf(0,1,2)
scipy.stats.norm.pdf(1,1,2)
```

Python

```
using Distributions

pdf(Normal(1,2),-1)
pdf(Normal(1,2),0)
pdf(Normal(1,2),1)
```

Julia